Using Federated Learning to Analyze Occupational Stress: Preserving Privacy and Identifying Patterns

Rodrigo Neves de Senna Marchiolli[1]

[1]Universidade Federal de Santa Catarina (UFSC)

rodrigo.nsmarchiolli@gmail.com

Abstract. *This study evaluated federated learning as a privacy-preserving approach for detecting and monitoring occupational stress The work combines a systematic literature review with a practical prototype that integrates wearable sensing, a mobile privacy gateway, anonymized communication, authenticated data submission, and local model training The review discusses the relevance of occupational stress, the role of wearable technologies and artificial intelligence in stress monitoring, and the privacy risks associated with centralized physiological data collection.*

For the practical implementation, a layered Internet of Medical Things (IoMT) architecture was developed in which ESP32-based wearable devices transmit physiological signals to an owner-authorized mobile application through authenticated Bluetooth Low Energy bonding The mobile application stores provisioning credentials securely, signs outgoing JSON payloads with HMAC-SHA256, a Hash-based Message Authentication Code using the SHA-256 hash function, and forwards authenticated requests through the Tor anonymity network to a local training node hosted at a healthcare institution Local model training was validated on the WESAD dataset using a regularized tabular Multilayer Perceptron (MLP), achieving a validation macro F1-score of 0.9565 for three-class stress classification The proposed model was compared with an original MLP baseline and alternative training strategies, using accuracy, precision, recall, macro F1-score, validation loss, and overfitting gap as evaluation criteria.

1. Introduction

Occupational stress, a significant concern in contemporary work environments, stems from the imbalance between job-related pressures and an individual's ability to cope with them, precipitating a range of detrimental consequences for personal health and organizational effectiveness Given that occupational stress is not a condition that can be cured through treatment but rather a risk factor requiring consideration of the problem's history, continuous monitoring and proactive management become paramount [Quick and Henderson 2016] The repercussions of occupational stress are multifaceted, impacting individual well-being through emotional distress and physical health problems, while also affecting organizations via diminished productivity and increased costs [Quick and Henderson 2016, Spector 2002] The current work environment's demands often clash with employees' physiological, emotional, and social needs, making it essential to create supportive work conditions that encourage growth and productivity while aiding individuals in their challenges [Mariotti 2015].

Wearable technology and artificial intelligence offer innovative pathways for detecting and managing occupational stress by continuously monitoring physiological signals and behaviors linked to stress responses [Quick and Henderson 2016]. The integration of wearable sensors, coupled with advanced data analytics, creates opportunities for real-time insights into an individual's stress levels, thereby enabling the implementation of timely and personalized intervention strategies [Alhejaili and Alomainy 2023, Alshamrani 2021]. Such interventions have the potential to mitigate the adverse effects of stress, fostering healthier and more productive work environments [Chen et al. 2021]. However, the traditional centralized data collection approach raises significant concerns regarding privacy and data security. The centralized storage and processing of sensitive employee data, such as physiological information and behavioral patterns, can expose individuals to potential breaches, unauthorized access, and misuse of their personal information. This poses substantial risks to both employee privacy and organizational data security. To address these concerns, alternative approaches that prioritize privacy and security while still enabling effective occupational stress analysis are needed.

Federated learning, a decentralized machine learning technique, addresses these privacy issues by allowing models to be trained locally on individual devices without sharing sensitive raw data [Can and Ersoy 2021] This approach holds significant promise for occupational stress analysis, as it enables the preservation of employee privacy and data security In contrast to traditional centralized machine learning methods, which involve the collection and storage of raw physiological and behavioral data, federated learning empowers individuals to train models on their own devices, sharing only model updates rather than the underlying personal information.

By leveraging federated learning, this study aims to explore its potential for analyzing occupational stress, emphasizing the dual objectives of accurate stress detection and the protection of data privacy The practical prototype extends this vision with a mobile privacy gateway that routes authenticated physiological data through the Tor network to local training nodes at healthcare institutions, ensuring that raw signals remain on-premises while only model updates participate in global aggregation.

Through a systematic literature review and practical implementation, this research intends to assess the effectiveness and advantages of federated learning compared to traditional centralized methods By combining local model training with a privacy-preserving communication path based on Tor and HMAC-SHA256, this work seeks not only to enhance privacy and data security, but also to maintain the precision and accuracy of stress detection models in realistic occupational health scenarios In this context, Tor hides the network origin of the request, while HMAC-SHA256 verifies message integrity and authorization at the application layer Additionally, the research aims to demonstrate the practical applicability and benefits of federated learning in addressing occupational stress, a critical issue with significant social, economic, and organizational implications.

This paper is organized into seven sections Section 2 details the methodology employed in the systematic review, investigating four databases: PubMed, IEEE Xplore, Scopus, and Web of Science; based on inclusion and exclusion criteria, a total of 11 scientific articles were selected Section 3 presents the main results of the systematic review, addressing the research questions and related discussions Section 4 covers the main challenges encountered and solutions implemented during the development of the

privacy-preserving prototype Section 5 presents the practical results, Section 6 discusses the implications of these findings, and Section 7 concludes the study.

2. Methodology

This research employed a mixed-methods approach, incorporating a systematic literature review and practical implementation to evaluate the effectiveness and security of federated learning for detecting and monitoring occupational stress.

The systematic literature review adhered to PRISMA guidelines, ensuring a rigorous and transparent process for identifying, selecting, and synthesizing relevant studies [Wang et al. 2024] The literature review aimed to explore the current state of research on federated learning, traditional machine learning, occupational stress, and wearable technologies, providing a comprehensive understanding of the field.

The review process consisted of clearly defined steps: (1) formulating research questions; (2) identifying relevant literature; (3) applying selection criteria to filter articles; (4) extracting and analyzing data; and (5) synthesizing and presenting the findings.

2.1. Formulating Research Questions

To guide the systematic literature review, the following research questions were established:

- **RQ1**: How can data security and privacy be ensured in the context of federated learning?
- **RQ2**: What types of machine learning algorithms are commonly employed in federated learning for occupational stress analysis?
- **RQ3**: What are the primary challenges and limitations of federated learning in real-world occupational settings?
- **RQ4**: How can federated learning be integrated with wearable technology for continuous and personalized occupational stress monitoring?
- **RQ5**: What are the main challenges and corresponding solutions found in applying federated learning for occupational stress monitoring?

2.2. Identifying Relevant Literature

Four academic databases were systematically searched: PubMed, IEEE Xplore, Scopus, and Web of Science, chosen for their extensive collections of peer-reviewed literature in health sciences, technology, and engineering.

Articles were selected based on clearly defined inclusion criteria as described in Table 1, such as peer-reviewed status, publication date (recent ten years), focus on federated learning, stress monitoring, and wearable technology Articles that did not meet these criteria or lacked sufficient methodological detail were excluded The selection process involved title and abstract screening, followed by full-text evaluation.

2.3. Applying Selection Criteria to Filter Articles

The search was conducted in Web of Science, Scopus, PubMed, and IEEE Xplore databases using the following search string to ensure relevance: ("occupational stress" OR stress) AND "federated learning" The specific search strings and adjustments for each database were as follows:

Table 1. Inclusion and Exclusion Criteria

Inclusion Criteria	Exclusion Criteria
Studies on Federated Learning and stress	Duplicate articles
Contribution to research questions	Articles not available in full text
Available in full version	Studies outside the scope
Peer-reviewed status	Review articles

- **Web of Science**
  - Search String: `ALL=(("occupational stress" OR stress) AND "federated learning")`
  - Adjustments: Display final publication year
  - Initial number of articles: 34
- **Scopus**
  - Search String: `TITLE-ABS-KEY ( ( "occupational stress" OR stress ) AND "federated learning" )`
  - Adjustments: Include only the document type *Article*
  - Initial number of articles: 10
- **PubMed**
  - Search String: `("occupational stress"[Title/Abstract] OR stress[Title/Abstract]) AND "federated learning"[Title/Abstract]`
  - Initial number of articles: 5
- **IEEE Xplore**
  - Search String: `("All Metadata":federated learning) AND ("All Metadata":occupational stress OR "All Metadata":stress)`
  - Initial number of articles: 41

The review process began with 90 articles from the Web of Science, Scopus, PubMed, and IEEE Xplore databases The RAYYAN tool was used to assist in the selection [Johnson and Phillips 2018] After removing 29 duplicates, 61 articles were screened Of these, 34 were excluded based on the title and abstract The remaining 27 were retrieved in full and assessed using the inclusion and exclusion criteria Sixteen articles were excluded for not meeting the criteria or being out of scope Ultimately, 11 articles were included in the systematic review The PRISMA flow diagram in Figure 1 details this selection process.

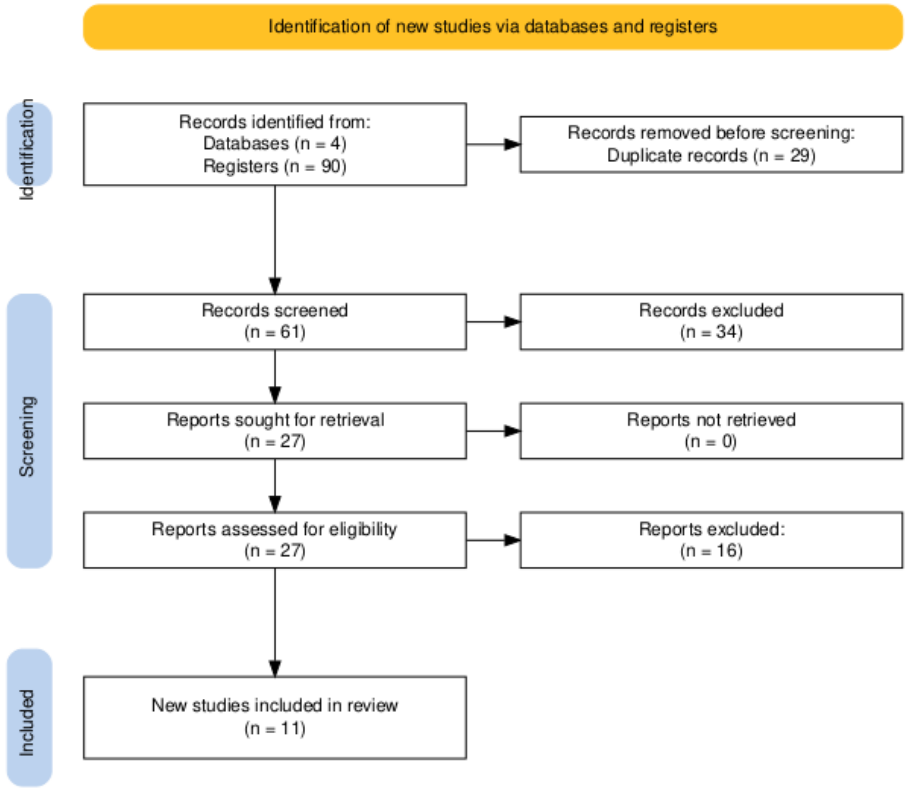


Figure 1. PRISMA flow diagram

3. Main Results and Discussion

3.1. *RQ1*: How can data security and privacy be ensured in the context of federated learning?

Federated learning (FL) provides inherent advantages for preserving data privacy by ensuring that sensitive information remains decentralized, a crucial requirement when dealing with confidential healthcare datasets [Sadilek et al. 2021] This approach minimizes the risks associated with data breaches and unauthorized access, aligning with stringent data protection regulations such as the General Data Protection Regulation (GDPR) and HIPAA [Bharathi et al. 2024] Several strategies enhance data security and privacy within federated learning frameworks, including differential privacy, homomorphic encryption, and secure multi-party computation [Xu et al. 2020].

In addition, regulations such as GDPR reinforce the importance of privacy-preserving technologies, highlighting the crucial role of FL in complying with specific data protection legislation In this context, complementary techniques have been explored to strengthen security and privacy in FL, including physical transmission layer security mechanisms, such as leveraging the unique properties of wireless channels to protect communications between local devices and central servers [Fauzi et al. 2022] By focusing on safeguarding both the client and server sides, federated learning addresses critical aspects such as convergence, data poisoning, scaling, and model aggregation [Ma et al. 2020].

Other advanced techniques have also been incorporated into FL to increase the protection of shared data For example, the use of Generative Adversarial Networks

(GANs) for data augmentation and Graph Attention Networks (GATs) have been explored in the literature to improve learning from distributed data while limiting the amount of information exchanged, further reducing the possibility of inferring sensitive information through reverse engineering. [Jiang et al. 2023].

Therefore, FL ensures data security and privacy mainly by keeping raw information locally, sharing exclusively the parameters of trained models and incorporating additional techniques such as Differential Privacy, advanced encryption and specific machine learning methods, which together provide robust protection against privacy attacks without compromising the performance of the developed systems.

3.2. *RQ2*: What types of machine learning algorithms are commonly employed in federated learning for occupational stress analysis?

There is no universal approach when developing artificial intelligence models applied to federated learning (FL) for occupational stress analysis Specialized literature suggests various algorithms commonly used due to their proven capability to handle specific federated learning challenges, such as data decentralization, information privacy, and device heterogeneity [Sadilek et al. 2021].

Among the most employed algorithms are *Artificial Neural Networks (ANN)*, *Random Forest (RF)*, *Support Vector Machines (SVM)*, *k-Nearest Neighbors (k-NN)*, and *Logistic Regression (LR)*. Each of these techniques has distinct characteristics and advantages suitable for practical applications in federated environments for occupational stress analysis.

A visual comparison summarizing the main algorithms discussed can be observed in Table 2.

3.2.1. Artificial Neural Networks (ANN)

Artificial Neural Networks are highly versatile algorithms inspired by the human brain's structure, capable of modeling complex relationships between input data (e.g., physiological signals acquired by wearable sensors) and outputs (e.g., detected stress levels) ANNs are particularly valued for their ability to extract non-linear patterns from multidimensional and heterogeneous data [Alahmadi et al. 2023] In federated learning contexts, they are widely adopted due to their flexibility in handling diverse physiological data types, such as electrocardiogram (ECG), electrodermal activity (EDA), and electromyography (EMG), along with effective adaptation to edge devices with computational resource constraints [Alahmadi et al. 2023].

Typical metrics: High accuracy (85-95%), high precision and recall.
Challenges: Requires large data volume and high local computational resources.

3.2.2. Random Forest (RF)

Random Forest is an ensemble method algorithm that combines results from multiple individual decision trees The main strength of this method lies in its robustness against

noisy data and its capacity to avoid overfitting, which is critically advantageous in federated learning environments where data quality can vary significantly among participants [Luong et al. 2023] Additionally, RF is particularly effective for physiological data, allowing accurate classification by consolidating multiple decisions from distributed devices, thereby reducing reliance on a single model or data source [Sadilek et al. 2021].

Typical metrics: High accuracy (80-93%), robust against noise.
Challenges: Potentially higher complexity when employing numerous trees on resource-limited devices.

3.2.3. Support Vector Machines (SVM)

SVM is widely recognized for effectively classifying complex data by establishing clear separation hyperplanes between classes This method consistently performs well in FL contexts for detecting physiological states related to stress, particularly when client-specific data availability is limited Nonetheless, SVM requires careful parameter tuning to handle non-linear and noisy data, frequently encountered in stress analysis from wearable sensors [Huang et al. 2024].

Typical metrics: Medium-high accuracy (75-90%), high precision.
Challenges: Complex hyperparameter tuning and sensitivity to outliers.

3.2.4. k-Nearest Neighbors (k-NN)

The k-NN algorithm is a simple yet effective method that classifies new data points based on the labels of their k nearest neighbors. In federated learning, its simplicity and low computational cost are particularly valuable, facilitating direct implementation on wearable devices with limited resources. However, challenges such as noisy data, local distribution heterogeneity, and class imbalance must be managed carefully to ensure reliable results. Advanced techniques, such as SMOTE or SMOTEENN, can mitigate imbalance issues that could compromise accuracy [Alahmadi et al. 2023].

Typical metrics: Medium accuracy (70-85%), good interpretability.
Challenges: High computational cost during prediction with large datasets.

3.2.5. Logistic Regression (LR)

Logistic Regression is a classic method for binary classification problems, modeling the probability that a data point belongs to a class based on explanatory variables. In FL, its primary advantage is simplicity, fast training, and low computational requirement, aspects that facilitate deployment on distributed devices. Nevertheless, it is sensitive to data balance and representativeness, demanding careful parameter tuning and additional strategies to prevent overfitting. Its applicability in federated environments is mainly explored when straightforward interpretation of variables is required [Alahmadi et al. 2023].

Typical metrics: Medium accuracy (70-85%), high computational efficiency.
Challenges: Sensitive to data imbalance.

3.2.6. Other Relevant Algorithms

Beyond the mentioned techniques, there is growing interest in using more complex algorithms, such as Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN), specifically designed for analyzing sequential and temporal data acquired by wearable sensors [Huang et al. 2024] Hybrid techniques and Multi-task Learning (MTL) methods have also been proposed to enhance federated model accuracy and robustness by sharing representations across related tasks[Benouis et al. 2023].

Ensemble learning techniques, like Gradient Boosting Machines and AdaBoost, are also highlighted for significantly improving predictive performance by combining multiple individual models, thus enhancing generalization capability in heterogeneous federated data scenarios [Alahmadi et al. 2023].

Table 2. Visual comparison of commonly employed machine learning algorithms in federated learning for occupational stress analysis.

Algorithm	Type	Accuracy	Noise Robustness	Computational Cost	Suitability for FL
ANN	Non-linear	High	Medium	High	High
Random Forest	Non-linear	High	High	Medium-High	High
SVM	Linear/Non-linear	Medium-High	Medium	Medium-High	Medium
k-NN	Non-linear	Medium	Low	Medium-Low	Medium
Logistic Regression	Linear	Medium	Low-Medium	Low	High

These descriptions and visual comparisons provide a comprehensive foundation, clearly illustrating each algorithm's technical characteristics and practical applicability in federated learning contexts for occupational stress detection.

3.3. *RQ3*: What are the primary challenges and limitations of federated learning in real-world occupational settings?

Federated learning, while promising, faces significant challenges in real-world occupational settings, primarily due to issues related to data heterogeneity, communication constraints, and system scalability [Jiang et al. 2024]. Data heterogeneity arises from variations in sensor types, data collection protocols, and individual physiological responses, making it difficult to train a globally consistent model [Lin et al. 2024].

Communication constraints, particularly in scenarios with limited bandwidth or intermittent connectivity, pose challenges for model aggregation and distribution [Li et al. 2020]. Addressing statistical heterogeneity, also known as non-IID (independent and identically distributed) data, is paramount in federated learning because the distribution of local data across different clients can vary significantly due to diverse occupational environments and individual differences, which impacts model convergence and performance.

Furthermore, the computational capabilities of edge devices used for local training can vary widely, creating additional challenges for deploying complex models and algorithms [Gahlan and Sethia 2023] Moreover, security and privacy considerations are paramount, necessitating robust mechanisms to defend against adversarial attacks such as

data poisoning and inference attacks, while simultaneously ensuring adherence to stringent data protection regulations such as GDPR and HIPAA, which mandate the safeguarding of sensitive health information [Sadilek et al. 2021].

The trustworthiness of federated learning systems is also a major concern, requiring careful attention to issues such as data quality, provenance, and bias detection to ensure reliable and equitable outcomes [Xu et al. 2020].

The lack of standardized benchmarks and evaluation metrics makes it difficult to compare the performance of different federated learning approaches and assess their suitability for specific occupational settings.

Therefore, addressing these challenges requires interdisciplinary collaboration involving experts in machine learning, cybersecurity, and occupational health, along with the development of innovative solutions that balance privacy, accuracy, and efficiency.

3.4. *RQ4*: How can federated learning be integrated with wearable technology for continuous and personalized occupational stress monitoring?

Federated learning can be seamlessly integrated with wearable technology to enable continuous and personalized occupational stress monitoring, offering numerous advantages over traditional centralized approaches By processing data locally on wearable devices, federated learning minimizes the need to transmit sensitive physiological data to a central server, thereby reducing the risk of data breaches and privacy violations [Liu et al. 2021] Wearable devices, equipped with sensors such as heart rate monitors and electrodermal activity sensors, continuously collect physiological data, which is then pre-processed and analyzed locally using lightweight machine learning models.

FL operates by training models locally on individual wearable devices or edge devices such as smartphones Each device can collect physiological signals such as BVP/PPG, EDA, respiration, skin temperature, ECG, or EMG, depending on the wearable configuration; these signals are commonly explored as indicators of stress-related physiological responses [Alahmadi et al. 2023].

These models are trained using algorithms such as federated averaging, which aggregates model updates from multiple devices to create a global model without directly accessing the raw data.

Additionally, the effectiveness of these technologies depends on their ability to accurately capture and interpret physiological signals indicative of stress [Almadhor et al. 2023].

Federated learning further preserves data privacy and promotes real-time distribution of models and learning by enabling devices to collaboratively train and evaluate a shared model while keeping personal data on site. Moreover, federated learning enables personalized stress models that adapt to individual physiological characteristics and work patterns, leading to more accurate and timely stress detection. The integration of federated learning with wearable technology requires careful consideration of factors such as device heterogeneity, communication bandwidth, and energy efficiency to ensure optimal performance and user experience.

3.5. *RQ5*: What are the main challenges and the corresponding solutions found in applying federated learning for occupational stress monitoring?

Applying federated learning to occupational stress monitoring presents several challenges related to data quality, privacy, and computational resources, which necessitate innovative solutions to ensure effective and reliable monitoring [Gahlan and Sethia 2023, Vyas et al. 2023] Data quality can be affected by sensor noise, missing values, and individual variations in physiological responses, which can impact the accuracy of stress detection models.

To address these challenges, robust data preprocessing techniques such as filtering, imputation, and normalization are essential to improve the quality and consistency of the data used for training federated learning models. Differential privacy mechanisms, such as adding noise to model updates, can be employed to protect sensitive data from inference attacks, but this must be done carefully to avoid compromising model accuracy.

One of the primary advantages of federated learning is its ability to maintain data privacy by processing data locally on user devices [Dai et al. 2021, Konečný et al. 2016].

The inherent architecture of federated learning prioritizes user privacy, as model training occurs on decentralized devices, and only model updates are shared with a central server [Gafni et al. 2022].

The heterogeneity of wearable devices and communication networks in occupational settings poses another challenge, requiring adaptive learning algorithms and communication protocols that can accommodate variations in computational resources and network bandwidth [Lin et al. 2024].

Despite these challenges, federated learning has been found to hold great promise in improving the detection and management of occupational stress.

4. Challenges Encountered and Solutions Implemented

During the practical phase of this research, the original architecture, in which ESP32 devices would communicate directly with a local MQTT gateway and then traverse the Tor network, was refined. The final prototype delegates network-intensive operations to a mobile application, which acts as an intermediary between wearable sensors and the local training node hosted at a healthcare institution. This decision reflects hardware constraints on embedded devices, which cannot efficiently run a full Tor client for continuous operation, while preserving the privacy goals outlined in the literature review.

4.1. Embedded Device and Network Constraints

A first challenge was the limited computational capacity of ESP32-based wearable devices. The ESP32 is a low-cost microcontroller commonly used in Internet of Things prototypes; it is appropriate for reading sensors and sending compact messages, but not for maintaining a full anonymized communication stack continuously. The implemented solution was to keep the wearable layer focused on sensing and to move Tor connectivity, QR provisioning, secure storage, and request signing to the smartphone. In this work, Tor is used as an anonymity network that hides the network origin of the request, QR provisioning is the process of configuring the application through a QR code, and request signing is the process of attaching a cryptographic proof that the message was produced

by an authorized client. This preserves the feasibility of the Internet of Medical Things (IoMT) layer while maintaining network-level privacy through the mobile gateway.

4.2. Credential Provisioning and Request Authentication

A second challenge was how to authorize incoming requests without exposing secrets in transit The solution was a QR-based provisioning workflow The local API generates credentials containing the hidden service address and a shared password A hidden service address is a Tor address ending in `.onion`, which allows the hospital-side service to be reached through the Tor network without exposing a conventional public IP address The mobile application scans this QR code, stores the credentials locally, serializes the outgoing JSON body, and computes an HMAC-SHA256 signature over the exact payload bytes HMAC-SHA256 is a keyed hash: it uses a shared secret and the message content to produce a signature that can be verified by the server, without sending the secret itself The server recomputes the signature and rejects requests with invalid signatures This mechanism adds integrity and authorization to the anonymous communication channel provided by Tor.

4.3. Local Model Design for WESAD Validation

A third challenge was choosing a model compatible with the actual feature representation used in this work. The WESAD preprocessing pipeline converts physiological signals into 30-second windows represented as tabular feature vectors, meaning that each observation is a row of numerical measurements rather than a raw time series. Therefore, the practical validation used a tabular Multilayer Perceptron (MLP), a neural network composed of fully connected layers, rather than a temporal model such as an LSTM, GRU, 1D-CNN, or transformer. The improved model includes regularization and training-stability mechanisms such as normalization inside the network, dropout, class-weighted loss, learning-rate scheduling, early stopping, gradient clipping, and light Gaussian noise augmentation. These mechanisms are described in more detail in Section 5, where they are connected to the experimental results.

4.4. Privacy-Oriented Feature Selection

The use of demographic attributes such as age, height, and weight created a privacy concern because these variables may act as quasi-identifiers. To evaluate this issue, a second version of the training script removed these features and retained only the physiological feature set. This demographics-free configuration produced a modest reduction in validation macro F1-score while preserving competitive performance, supporting the feasibility of a more privacy-conscious deployment.

4.5. Federated Learning Integration Scope

The current prototype validates the data ingestion, authentication, anonymized transport, local WESAD training, and software integration path for Flower-based federated learning Flower is the framework used to coordinate federated training rounds: it sends model parameters to local clients, receives locally updated parameters, and aggregates them into a new global model The results reported in this paper should therefore be interpreted as validation of the local client-side training pipeline and of the surrounding privacy-preserving communication architecture They should not yet be interpreted as a completed multi-institutional benchmark with independent real hospital datasets.

5. Results

This section presents the practical outcomes of the project, organized according to the evolved system architecture. The implementation combines wearable physiological sensing, a mobile privacy gateway, network anonymization through Tor, application-layer authentication via HMAC, local data storage at the institutional node, federated training with Flower, and model redistribution to the mobile application. The WESAD dataset is used as a validation benchmark for the stress classification pipeline.

To make the results self-contained, Table 3 summarizes the main technical terms used throughout this section and their role in the implemented prototype.

Table 3: Main technical terms used in the Results section.

Term	**Meaning**	**Role in this project**
IoMT	*Internet of Medical Things*; connected medical or health-related sensing devices.	Sensing layer composed of wearable physiological sensors connected to the ESP32.
ESP32	Low-cost microcontroller with wireless communication capabilities.	Local sensing unit controller that collects signals from wearable sensors and sends them to the mobile application through authenticated Bluetooth Low Energy bonding.
Bluetooth Low Energy (BLE) bonding	Persistent authenticated pairing between a peripheral device and a central device.	Restricts local sensor access to the smartphone that completed authenticated enrollment with the sensing unit.
BVP, EDA, TEMP, Resp	Physiological signals related to blood volume pulse, electrodermal activity, skin temperature, and respiration.	Basis for stress-related feature extraction and classification.
Mobile Gateway	Smartphone application that mediates communication between the sensing layer and the institutional node.	Receives ESP32 data, stores credentials, signs requests, routes traffic through Tor, and supports user interaction.
Tor	An anonymity network based on onion routing.	Hides the network origin of requests sent by the mobile application to the hospital-side hidden service.
HMAC-SHA256	Cryptographic message authentication mechanism based on a shared secret and SHA-256 hashing.	Verifies request authenticity and integrity before physiological data are accepted by the local API.
PostgreSQL	Relational database management system.	Stores authenticated physiological windows locally inside the institutional environment.

Table 3: Main technical terms used in the Results section. Continued.

Term	Meaning	Role in this project
Flower	Federated learning framework for coordinating distributed model training.	Implements communication between local training clients and the global federated server.
FedAvg	*Federated Averaging*; aggregation rule for combining model updates from multiple clients.	Aggregates local model parameters into a global model without centralizing raw physiological data.
ONNX	Open Neural Network Exchange; interoperable format for machine learning models.	Exports the updated global model for mobile-oriented inference and redistribution.

5.1. Evolved System Architecture

The practical prototype consolidates six operational layers, as summarized in Table 4 and Figure 2. Compared with the initial design, the most significant change is the insertion of a mobile application between the ESP32-based wearables and the local training node hosted by the healthcare institution. Rather than requiring each embedded device to establish an anonymized connection, physiological data are first received by the smartphone, which then performs secure forwarding through the Tor network toward the institution's local server.

Table 4. Operational layers of the evolved IoMT architecture for occupational stress monitoring.

Layer	Component	Primary Function	Privacy Measure
IoMT Devices	ESP32 and wearable sensors	Collection of physiological sensor signals	Device pseudonyms; authenticated BLE bonding; single-owner access control
Mobile Gateway	React Native application	Reception of sensor data and secure forwarding	Local secure storage; Tor client on the smartphone
Network Anonymization	Tor hidden service (`.onion`)	Concealment of the network path between the phone and the institution	Onion routing; origin IP masking
Application Authentication	HMAC-SHA256 middleware	Request integrity and client authorization	Shared secret; signed JSON payload
Local Training Node	Hospital server / edge node	Local persistence, preprocessing, and Flower client execution	Raw physiological data remain on-premises
Federated Coordination	Flower server using gRPC	Global model aggregation through FedAvg	Only model updates are exchanged

The end-to-end flow proceeds as follows:

1. Wearable sensors connected to a single ESP32 collect physiological windows and transmit them to the authorized mobile device through an authenticated Bluetooth Low Energy bond.
2. The mobile application stores configuration credentials locally and prepares a signed HTTP request.
3. The request traverses the Tor network and reaches the hidden service exposed by the local training node.
4. The local API validates the HMAC signature and persists the received data in the institution's database.
5. During a federated training round, the local Flower client reads these stored records, trains locally, and returns only updated model parameters to the global server.

Figure 2 provides a visual overview of these layers and their interactions.

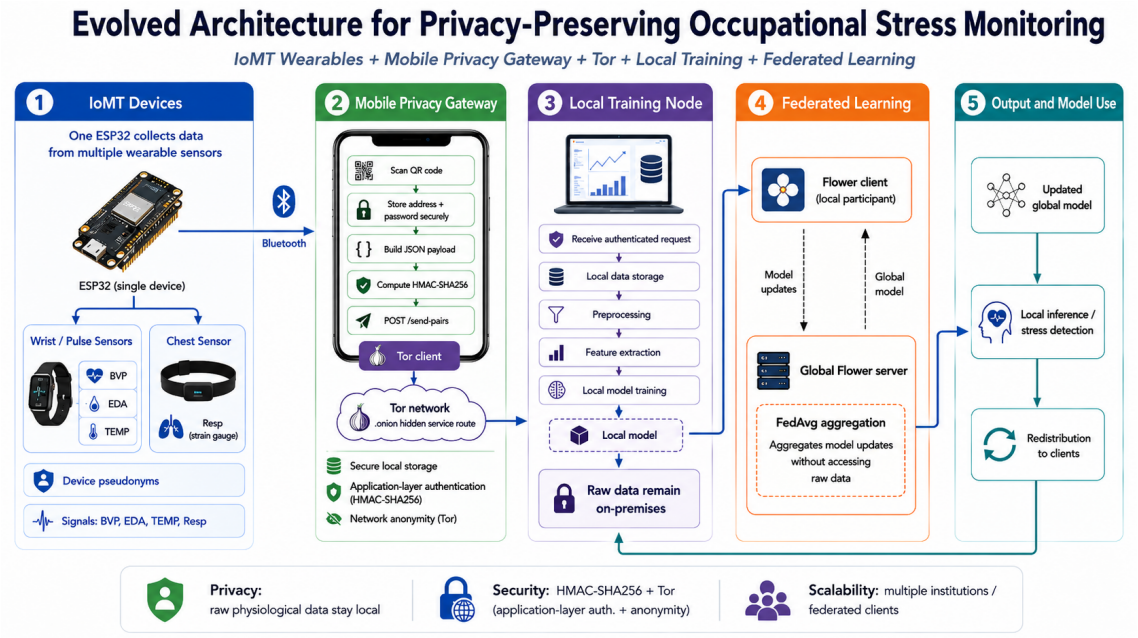


Figure 2. Evolved IoMT architecture for privacy-preserving occupational stress monitoring. Physiological data flow from a single ESP32 connected to wearable sensors to the owner-authorized mobile gateway through authenticated Bluetooth Low Energy bonding, then traverse Tor with HMAC-authenticated requests, and are processed locally at the healthcare institution before federated model aggregation.

5.2. Mobile Application as Privacy Gateway

The mobile application was developed as an Expo/React Native prototype with native modules for Tor connectivity, camera-based QR code scanning, secure credential storage, and communication with the ESP32-based sensing unit. Expo/React Native was used because it supports mobile interface development in JavaScript/TypeScript while still allowing native modules for capabilities such as Bluetooth and Tor connectivity. This design addresses a central limitation identified during prototyping: executing a Tor daemon directly on ESP32 hardware is computationally prohibitive for continuous operation. By

contrast, modern smartphones provide sufficient resources to run a native Tor client while maintaining an intuitive interface for credential provisioning, local request preparation, and secure data submission.

In the proposed workflow, a single ESP32 collects physiological signals from multiple wearable sensors Wrist or pulse sensors provide BVP, EDA, and skin temperature measurements BVP, or blood volume pulse, captures changes in blood flow and is related to cardiovascular response EDA, or electrodermal activity, captures changes in skin conductance and is commonly associated with sympathetic nervous system activation The chest sensor provides respiratory data through a strain gauge, which detects expansion and contraction during breathing The ESP32 transmits these physiological readings to the mobile application through authenticated Bluetooth Low Energy bonding The smartphone then acts as a privacy gateway between the local sensing layer and the hospital-side training infrastructure.

The mobile gateway pipeline comprises three main stages:

1. **Credential provisioning:** the local training node API generates a QR code containing a JSON object with the `.onion` service address and a shared password. The user scans this QR code with the mobile application, and the credentials are stored securely on the device.

2. **Local request preparation:** the application receives physiological data from the ESP32, serializes the readings into a JSON payload, computes an HMAC-SHA256 signature using the shared password, and attaches the resulting digest to the request through the `x-signature` header.

3. **Secure submission and validation:** the application sends a `POST` request to `/send-pairs` through the Tor network. The hospital API verifies the HMAC signature and persists valid data locally.

Training is not started by the HTTP request itself; it is executed later by the local Flower client when the federated server coordinates a training round. End-to-end tests confirmed the expected validation behavior: requests with invalid signatures are rejected with HTTP status `401`, whereas correctly signed requests are accepted and persisted with HTTP status `201`. Figure 3 summarizes this pipeline, emphasizing the distinction between application-layer authentication provided by HMAC-SHA256 and network-layer anonymization provided by Tor.

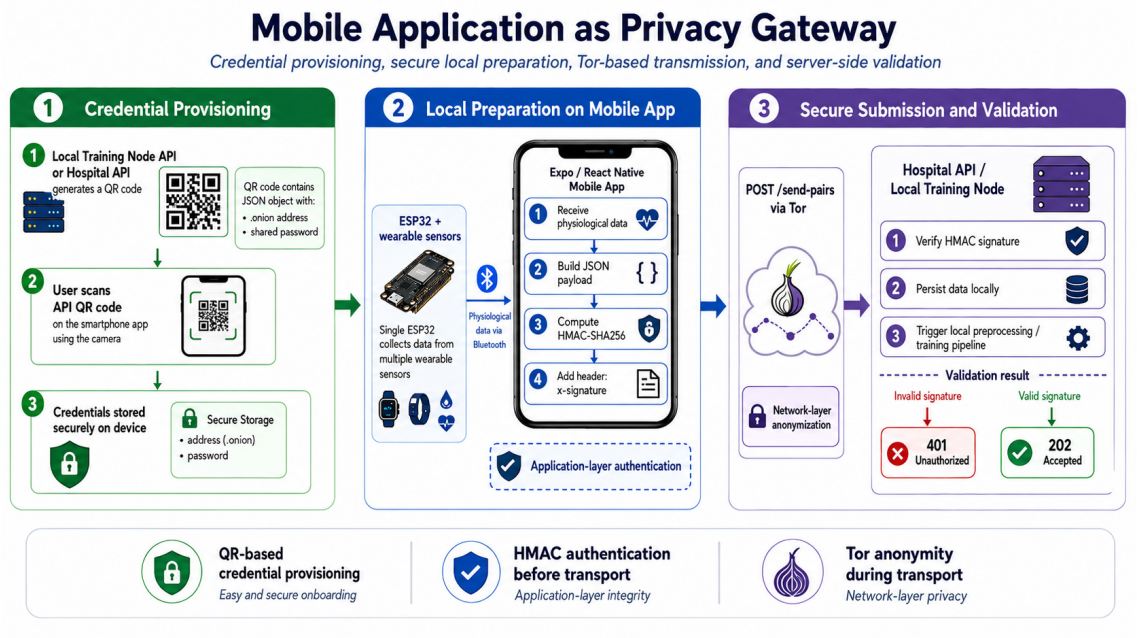


Figure 3. Mobile application pipeline as a privacy gateway. The local training node provisions credentials through a QR code, the mobile app stores the `.onion` address and shared password securely, receives physiological data from the ESP32 through authenticated Bluetooth Low Energy bonding, signs the JSON payload with HMAC-SHA256, and submits the request through Tor to the hospital API for signature validation and local persistence.

5.3. Bluetooth Pairing and Sensor Ownership

The communication between the sensing unit and the mobile gateway was also protected through a device-ownership mechanism at the Bluetooth Low Energy layer. In this design, the sensing unit operates as a peripheral device and the smartphone acts as the central device. The first smartphone that completes authenticated pairing with a previously unclaimed sensing unit becomes its trusted owner. Subsequent access to physiological readings and acknowledgement commands is accepted only when the active Bluetooth connection corresponds to this bonded owner.

This mechanism was implemented using authenticated Bluetooth Low Energy bonding with Secure Connections, man-in-the-middle protection, and passkey-based pairing. The bonding material is persisted in non-volatile memory, allowing the sensing unit to recognize the same smartphone after power cycles without requiring a new enrollment procedure. The authorization decision is therefore based on the cryptographic identity established during bonding, rather than on public or unstable identifiers such as the advertised device name, the observed Bluetooth address, or an application-level token.

The Bluetooth layer applies defense in depth. First, the sensing unit is configured to maintain only one active Bluetooth connection and one persistent bond. Second, sensitive operations require an authenticated and encrypted link. Third, before returning physiological windows or accepting acknowledgement commands, the firmware verifies that the current connection is encrypted, authenticated, bonded, and associated with the stored owner identity. If these conditions are not satisfied, the operation is rejected and the connection may be terminated. This prevents a second nearby smartphone from reading

buffered physiological data or sending false acknowledgements that could delete records before the legitimate mobile gateway processes them.

The adopted strategy complements, rather than replaces, the later privacy layers of the architecture. Bluetooth ownership protects the local sensing channel between the wearable device and the smartphone; HMAC-SHA256 authenticates submissions from the mobile gateway to the institutional node; Tor hides the network origin of those submissions; and federated learning prevents raw institutional data from being centralized. Together, these mechanisms establish a layered privacy model across the sensing, mobile, network, application, and learning stages.

5.4. Mobile Application Interface and User Workflow

Beyond acting as a privacy gateway, the mobile application was designed to provide a clear and usable interface for monitoring physiological windows, reviewing model predictions, correcting labels, and managing the connection with the sensing device. This interface is important because occupational stress monitoring should not operate as a black-box process. Instead, the user must be able to understand the current system state, inspect predictions, correct possible errors, and control when locally prepared data are submitted to the server.

The prototype interface is organized into four main screens, summarized in Table 5: Dashboard, History, Reports, and Device. Together, these screens support three practical goals: real-time interpretability, local data quality control, and operational reliability.

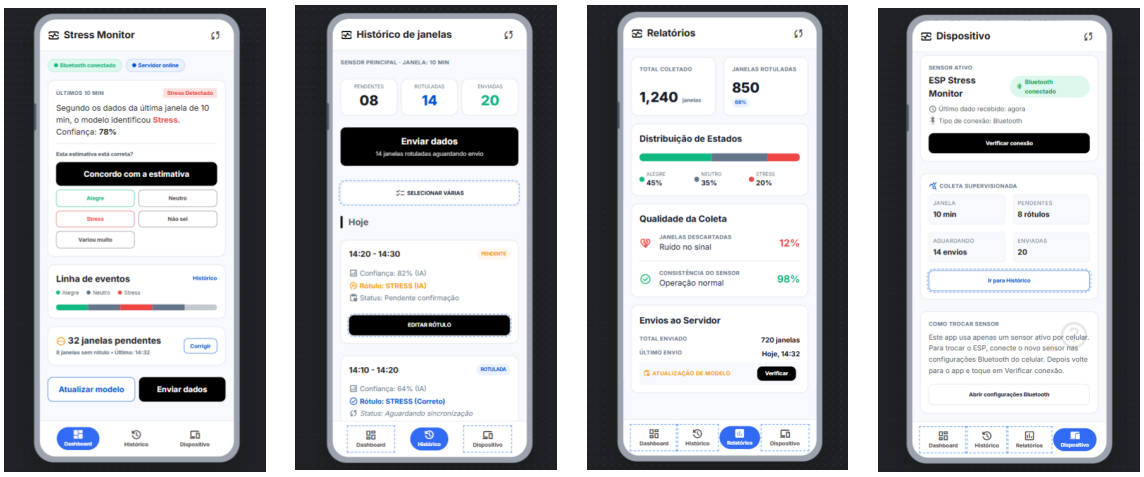
Table 5. Main screens and user-facing functions of the mobile prototype.

Screen	Purpose	Main information and actions
Dashboard	Real-time monitoring overview	Shows Bluetooth status, server status, latest prediction, confidence score, event timeline, and allows the user to confirm or correct the latest estimate.
History	Traceability and label correction	Lists physiological windows with time interval, model confidence, predicted or corrected label, synchronization state, individual editing, discarding, reopening, and batch labeling.
Reports	Longitudinal interpretation	Consolidates labeled windows into summaries, such as label distribution, stress-related proportions, and temporal trends.
Device	Sensing-layer management	Shows active sensor, Bluetooth state, last received data, connection type, and actions for checking or changing the device.

The Dashboard screen provides a high-level view of the monitoring process, while the History screen provides traceability over the physiological windows generated by

the system. The Reports screen is intended to consolidate labeled windows into interpretable summaries over time, and the Device screen centralizes the operational status of the sensing layer. This separation helps distinguish communication problems from model or server-side issues and reduces the effort required to correct or review consecutive windows that represent the same affective state.

Figure 4 illustrates the main navigation screens of the mobile prototype.



Dashboard | History | Reports | Device

Figure 4. Main navigation screens of the mobile prototype.

5.5. Network Anonymization through Tor

The Tor network provides *topological anonymity* by routing traffic through a sequence of relay nodes using onion encryption In practical terms, this means that the destination service does not directly observe the real IP address of the mobile device In this architecture, Tor complements, but does not replace, transport-layer protections such as TLS Its primary role is to obscure the network origin of the mobile gateway before data reach the local training node, thereby reducing the risk of correlating a participant's IP address with a specific healthcare institution or individual session.

Tor protects three aspects of the communication path: (i) the real IP address of the mobile gateway as observed by the destination service; (ii) the observable network route between origin and destination; and (iii) direct association between a network endpoint and a logical participant in the system. However, Tor alone does not protect the semantic content of messages if payloads are transmitted in plaintext, nor does it authenticate legitimate clients. Poorly designed topic names or unencrypted JSON fields could still reveal sensitive information even when routed through Tor. For this reason, the architecture treats Tor as one layer within a broader privacy strategy that also includes pseudonymous device identifiers, payload signing, and federated learning.

In operational terms, the mobile client connects to a hidden service address (`.onion`) exposed by the local training node's API. Traffic is forwarded through the native Tor module rather than a conventional direct HTTP connection, ensuring that the institution receives data without observing the patient's real network identity at the application boundary. This separation between *network identity* and *logical device identity*

is particularly relevant in occupational health scenarios, where employees may hesitate to share stress-related physiological data if they fear employer-level network monitoring. As illustrated in Figure 2, Tor operates at Layer 3 and therefore complements, rather than replaces, the mobile gateway and authentication mechanisms that precede and follow it in the pipeline.

5.6. Application-Layer Authentication with HMAC-SHA256

While Tor addresses anonymity at the network layer, it does not verify whether an incoming request originates from an authorized federated client To close this gap, the system employs HMAC-SHA256 (*Hash-based Message Authentication Code*) at the application layer HMAC combines a shared secret key with the message content through a cryptographic hash function, producing a deterministic signature that allows the server to verify both authenticity and integrity of each request.

The authentication contract operates as follows. The local training node generates a QR code containing two fields: `address`, corresponding to the `.onion` hidden service URL, and `password`, a shared secret known only to the institution and the authorized mobile client. When submitting data, the mobile application computes:

$$\text{signature} = \text{HMAC-SHA256}(\text{password}, \text{raw JSON body}) \tag{1}$$

The resulting digest, encoded as hexadecimal, is sent in the HTTP header `x-signature`. The server recomputes the signature from the raw request body and rejects any mismatch. Critically, the signature is computed over the exact byte sequence transmitted in the POST body; any alteration to field ordering, whitespace, or numeric formatting would invalidate the authentication.

Table 6 summarizes the separation of responsibilities between Tor and HMAC-SHA256. Tor hides the network route, while HMAC-SHA256 authenticates the request and protects payload integrity at the application layer.

Table 6. Separation of responsibilities between Tor and HMAC-SHA256.

Mechanism	Protects	Does not protect	Layer
Tor	Network origin and communication path visibility.	Payload integrity and client authorization.	Network / routing layer.
HMAC-SHA256	Request authenticity and payload integrity.	Network origin anonymity.	Application layer.

HMAC was chosen for three complementary reasons aligned with the privacy objectives of this project. First, it provides *authentication without credential exposure*: the shared password never travels in the request headers or body, only its derived signature does, reducing the risk of secret interception on compromised channels. Second, it guarantees *payload integrity*, ensuring that physiological records cannot be tampered in transit without detection. Third, it establishes a lightweight *authorization boundary* at the local training node, allowing the hospital to accept data only from clients provisioned through

the QR code workflow, independent of the anonymized network path established by Tor. Together, Tor and HMAC implement a defense-in-depth strategy: Tor conceals *who sent* the traffic at the network level, while HMAC confirms *who is authorized* to submit data at the application level, as summarized in Figure 3.

5.7. Local Training Node at the Healthcare Institution

The local training node represents the federated client deployed at a healthcare institution registered in the network. Conceptually, this node corresponds to a hospital or occupational health unit that maintains custody over its patients' physiological records while contributing to a collaboratively trained global model. The node hosts three subsystems: a Tor-accessible API for secure data ingestion, a local PostgreSQL database for storing validated sensor windows, and a Python Flower client responsible for preprocessing and local model training during federated rounds.

Upon receiving authenticated payloads, the local API stores data locally in an append-only sensor table. The records correspond to 30-second temporal windows, from which statistical and derived features are extracted before persistence or during preprocessing. This window-based representation transforms continuous sensor streams into tabular observations suitable for lightweight neural classification, while keeping raw and processed records within the institution's administrative boundary. When Flower starts a federated round, the local client reads the stored records, trains the model locally, and returns model parameters rather than physiological traces to the global federated server.

5.8. Feature Extraction and Preprocessing Pipeline

The classification pipeline was validated using the WESAD dataset, a publicly available multimodal corpus for stress and affect recognition with wearable sensors WESAD is not identical to the intended occupational deployment, but it provides a controlled benchmark with physiological signals compatible with the prototype: blood volume pulse (BVP), electrodermal activity (EDA), respiration, and skin temperature.

5.8.1. Window-Level Representation

The model does not receive raw time series directly. Instead, continuous physiological signals are transformed into 30-second windows, and each window becomes one tabular sample. This design choice is central to the rest of the training pipeline: after feature extraction, one row represents one physiological interval, not an entire participant or a complete recording session.

The target variable has three classes inherited from the WESAD protocol: *amusement* (class 0), *baseline* (class 1), and *stress* (class 2). The training and validation partitions used in the local experiments contain 805 and 202 window-level samples, respectively. Because the split was performed at the window level, results should be interpreted as a controlled development benchmark; a subject-level split remains necessary for a stronger out-of-subject generalization test.

5.8.2. Meaning of the Input Features

The complete feature set contains 29 columns. Of these, 26 are physiological descriptors and three are demographic attributes. The physiological columns summarize the behavior of each sensor inside a 30-second window, using statistics such as mean, standard deviation, minimum, and maximum, plus a small number of domain-specific derivatives.

Table 7. Feature groups used in the WESAD tabular training pipeline.

Group	Features	Interpretation
BVP	mean, std, min, max, peak frequency	Blood volume pulse magnitude and dominant rhythmic component.
EDA phasic	mean, std, min, max	Fast electrodermal responses associated with short-term arousal changes.
EDA SMNA	mean, std, min, max	Sudomotor activity estimate related to sympathetic nervous activation.
EDA tonic	mean, std, min, max	Slower baseline component of skin conductance.
Respiration	mean, std, min, max	Breathing pattern summarized over the window.
Temperature	mean, std, min, max, slope	Skin temperature level, variability, range, and local trend.
Demographics	age, height, weight	Static participant attributes used only in full local-reference models.

The demographic columns were kept only in the full local-reference configuration because they were present in the original tabular dataset. They were removed from the privacy-oriented configurations because age, height, and weight may behave as quasi-identifiers and may also encourage the model to exploit participant-specific shortcuts rather than physiological patterns that generalize across subjects.

5.8.3. Preprocessing Contract

All configurations apply type conversion and median imputation before training. The medians are computed only from the training split and then reused for validation, preventing validation statistics from leaking into model fitting. This is important because leakage can produce overly optimistic validation metrics.

The first local experiments also used `StandardScaler`. This transformation subtracts the training-set mean of each feature and divides by the corresponding training-set standard deviation. In a centralized experiment, this usually helps neural networks because all features arrive at a comparable numeric scale. For the deployment-oriented federated prototype, however, this scaler became an integration risk. The transformation is not fixed by the code alone; it depends on means and standard deviations estimated from a specific training dataset. To reproduce the same representation everywhere, the exact scaler parameters would need to be persisted, distributed, and applied identically in the embedded and simulated data flow, in each local training client, in server-side validation, and in mobile inference.

Table 8 summarizes the resulting engineering trade-off. During integration, the scaled contract was not reliable: when the model trained with scaling received unscaled or differently scaled inputs, Flower validation dropped to approximately 0.4 macro F1. For this reason, the selected integration pipeline uses the no-scaler contract.

Table 8. Comparison between the scaled local-reference pipeline and the selected no-scaler integration pipeline.

Pipeline	Advantage	Limitation
Scaled local-reference	Higher local validation performance because features are normalized before training.	Requires persisting and reproducing the exact scaler statistics across training, Flower, server validation, and mobile inference.
No-scaler integration	Uses original feature scale and is easier to reproduce across the full deployed system.	Lower local validation performance compared with the best scaled configuration.

In this work, *no scaler* means that no feature standardization is applied after type conversion and median imputation. This choice reduces local validation performance, but it removes a fragile external dependency from the end-to-end system.

5.9. Model Architecture and Training Configuration

Given the tabular representation described above, the project uses a Multilayer Perceptron (MLP) rather than a temporal model such as an LSTM, GRU, 1D-CNN, or transformer. Those architectures are appropriate when the model receives raw sequences. Here, the temporal signal has already been summarized into a fixed-length feature vector, so a fully connected network is a simpler and more coherent fit.

5.9.1. Baseline and Proposed TabularMLP

The original baseline was a simple MLP with two hidden layers and ReLU activations This baseline is useful because it measures how much of the classification signal is already present in the extracted features The proposed model, named *TabularMLP*, keeps the same family of fully connected networks but adds regularization and training-stability mechanisms Table 9 summarizes the main architectural differences.

Table 9: Comparison between the baseline MLP and the proposed TabularMLP.

Aspect	Original baseline MLP	Proposed TabularMLP
Hidden layers	Two fully connected layers.	Three fully connected layers with dimensions 128, 128, and 64.
Activation	ReLU.	GELU.
Normalization	Not used.	BatchNorm1d after each hidden linear layer.
Regularization	Limited.	Dropout with probability 0.25 and AdamW weight decay.
Output	Three class logits.	Three class logits.

The final layer returns three raw logits, one for each class; no final softmax is included because PyTorch's cross-entropy loss expects logits.

5.9.2. Training Strategy

The training procedure was designed for a small tabular physiological dataset, where two risks are especially relevant: overfitting and class imbalance. Overfitting occurs when the model adapts too closely to the training windows and loses generalization capacity on validation windows. Class imbalance occurs when one affective state has more samples than the others, which can make accuracy misleading because a model may perform well on the majority class while failing on less frequent classes.

Optimization was performed with AdamW, using an initial learning rate of 2×10^{-3} and weight decay of 10^{-3} AdamW is an adaptive optimization method that updates each model parameter according to the observed gradients while applying weight decay as a separate regularization term In practical terms, this discourages excessively large weights and helps reduce overfitting in small datasets Training was performed in mini-batches of 64 samples, allowing the model to learn from shuffled subsets of the training data rather than recalculating each update from the full dataset at once.

The loss function was class-weighted cross-entropy Cross-entropy is suitable for this task because the model outputs one score for each target class, corresponding to *amusement*, *baseline*, and *stress* The class weights are computed from the training-label frequencies, assigning a larger penalty to errors in less frequent classes This prevents the optimization process from being dominated only by the most common affective state An additional sampling-based rebalancing strategy was also evaluated, in which less frequent classes are sampled more often during training It was not adopted as the default strategy because combining this sampling policy with class-weighted loss can overcompensate minority classes in small datasets.

Model selection was based on validation macro F1-score, not on accuracy Macro F1 computes the F1-score independently for each class and then averages the three values, giving equal weight to all affective states This criterion is more appropriate for the WESAD experiments because the objective is not merely to maximize the number of correct predictions, but to maintain balanced performance across *amusement*, *baseline*, and *stress* The model retained for comparison is therefore the one obtained at the epoch with the highest validation macro F1-score, subject to a minimum improvement threshold of 10^{-4}.

The training procedure also includes mechanisms to control convergence The learning rate is reduced by a factor of 0.5 when validation macro F1-score stops improving for 15 consecutive epochs, with a minimum learning rate of 10^{-6} Early stopping interrupts training when validation macro F1-score does not improve for 50 epochs, avoiding unnecessary optimization after the model has saturated Finally, gradient clipping limits the gradient norm to 1.0, reducing the risk of unstable parameter updates during backpropagation.

Table 10. Main training hyperparameters used in the TabularMLP experiments.

Component	Value	Purpose
Optimizer	AdamW	Adaptive optimization with decoupled weight regularization.
Learning rate	2×10^{-3}	Initial magnitude of parameter updates.
Weight decay	10^{-3}	Penalizes overly large weights to reduce overfitting.
Batch size	64	Number of samples used per optimization step.
Maximum epochs	300	Upper bound on the number of training passes.
Learning-rate policy	Reduction after validation stagnation	Refines optimization when validation macro F1 stops improving.
Early stopping patience	50 epochs	Stops training after prolonged lack of validation improvement.
Gradient clipping	1.0	Limits abrupt gradient updates.

5.9.3. Augmentation and Class Rebalancing

The scaled models use online Gaussian noise augmentation after standardization In this setting, a noise value such as 0.03 has the same meaning for all features because the scaler places them in approximately comparable units.

The no-scaler models cannot safely use the same absolute noise. Without standardization, each feature has its own unit and magnitude; a fixed perturbation can be irrelevant for a large-scale feature and destructive for a small-scale feature. Therefore, the selected no-scaler experiments use online per-feature robust noise. The noise magnitude is proportional to a robust scale estimated from the training split, approximately $\mathrm{IQR}/1.349$, and augmented values are clipped between the 1st and 99th training quantiles. The augmentation probability is 0.5 and the noise fraction is 0.02.

This augmentation is online: it does not permanently expand the stored dataset. Instead, batches may receive different controlled perturbations across epochs. Sampling-based class rebalancing was also evaluated as an additional class-imbalance mechanism, but it did not improve the selected no-scaler variant.

Figures 5 and 6 show how macro F1-score and validation loss evolved across training epochs for the evaluated configurations.

Figure 6 complements this analysis by tracking validation loss throughout training. Validation loss measures how costly the model's predictions are on unseen validation samples; lower values indicate that the predicted probabilities are closer to the correct labels. Across the best-performing scaled configurations, loss decreases during the initial epochs and then stabilizes as learning-rate reduction and early stopping intervene. The *New full* model achieves the lowest validation loss at its best checkpoint (0.1454 at epoch 77), whereas the original baseline retains a higher loss (0.2279 at epoch 38) despite reaching its F1 peak earlier. The selected no-scaler configuration has a higher validation loss (0.3526 at epoch 117), which reflects the cost of removing feature standardization. This result is acceptable for the prototype because the no-scaler model is more consistent with the real data path used by the embedded sensing flow, simulated data flow, and federated learning components.

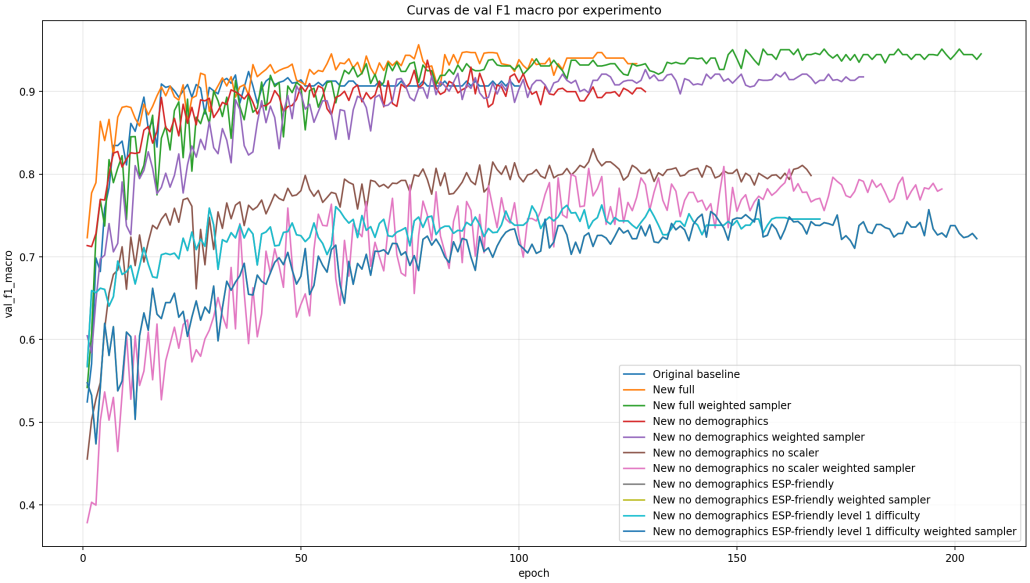


Figure 5. Validation macro F1-score curves for all WESAD training configurations across epochs. The scaled *New full* model reaches the highest local-validation peak (0.9565 at epoch 77), while the selected no-scaler variant favors deployment consistency.

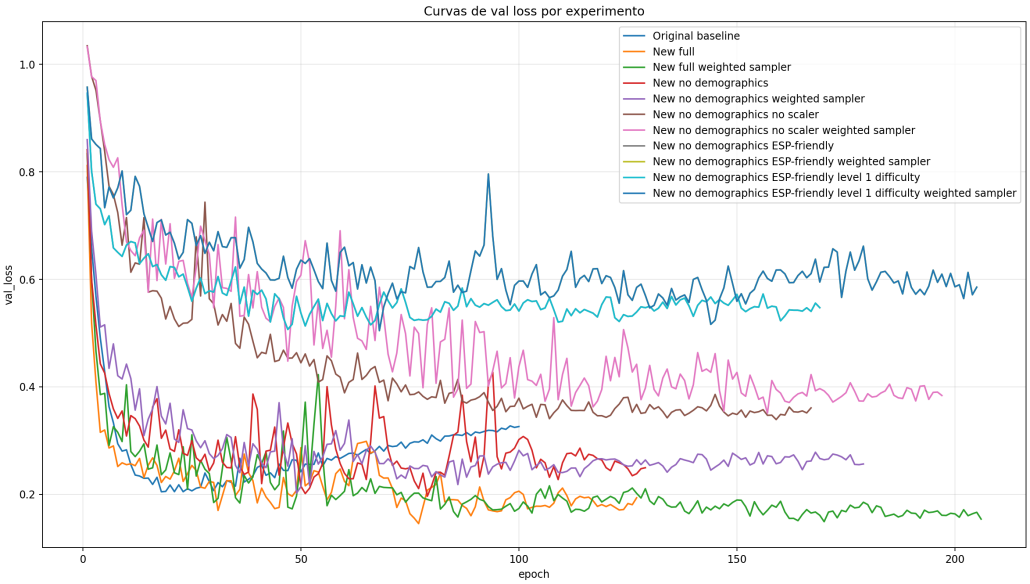


Figure 6. Validation loss curves for all WESAD training configurations. Lower and more stable validation loss at the best epoch indicates better fit to unseen windows; the scaled *New full* model reaches the lowest local-validation loss.

5.10. Experimental Classification Results

Eleven training configurations were evaluated to separate model quality, privacy-oriented feature selection, preprocessing reproducibility, and embedded feasibility The configurations can be read as four experimental questions, summarized in Table 11.

Table 11. Experimental questions represented by the evaluated WESAD configurations.

Configuration family	Question addressed	Reason for evaluation
Original baseline	Is a simple MLP already sufficient for the extracted features?	Establishes a reproducible reference point.
New full	What is the best local validation performance with all features and standardization?	Measures the upper local-performance reference.
No demographics / no scaler	What is the cost of privacy-oriented and deployment-oriented constraints?	Removes quasi-identifiers and avoids scaler synchronization across services.
ESP-friendly	What is the cost of using features easier to compute in an embedded path?	Estimates feasibility for simplified sensor-side processing.

Table 12 summarizes the best validation performance achieved by each configuration, ranked by validation macro F1-score Figure 7 compares accuracy, precision, recall, and macro F1 at the best epoch of each experiment.

Table 12. Best validation metrics per training configuration on the WESAD 30-second window dataset.

Configuration	Features	Best Epoch	Val F1 Macro	Val Accuracy	Val Precision	Val Recall
New full (best local)	29	77	**0.9565**	0.9653	0.9605	0.9531
New + weighted sampler	29	156	0.9513	0.9604	0.9472	0.9557
New without demographics	26	79	0.9378	0.9455	0.9310	0.9458
New no demo + weighted sampler	26	129	0.9265	0.9356	0.9177	0.9374
Original baseline MLP	29	38	0.9241	0.9406	0.9406	0.9127
New no demo no scaler (selected)	26	117	0.8307	0.8465	0.8194	0.8546
New no demo no scaler + weighted sampler	26	147	0.8093	0.8218	0.7994	0.8459
ESP-friendly + weighted sampler	13	155	0.7693	0.7921	0.7638	0.7991
ESP-friendly level 1 + weighted sampler	13	155	0.7693	0.7921	0.7638	0.7991
ESP-friendly	13	119	0.7627	0.7921	0.7559	0.7794
ESP-friendly level 1	13	119	0.7627	0.7921	0.7559	0.7794

The *New full* configuration achieved the highest local validation macro F1-score, 0.9565 at epoch 77, outperforming the original baseline score of 0.9241 by approximately 3.2 percentage points Its validation loss was also the lowest among the evaluated models

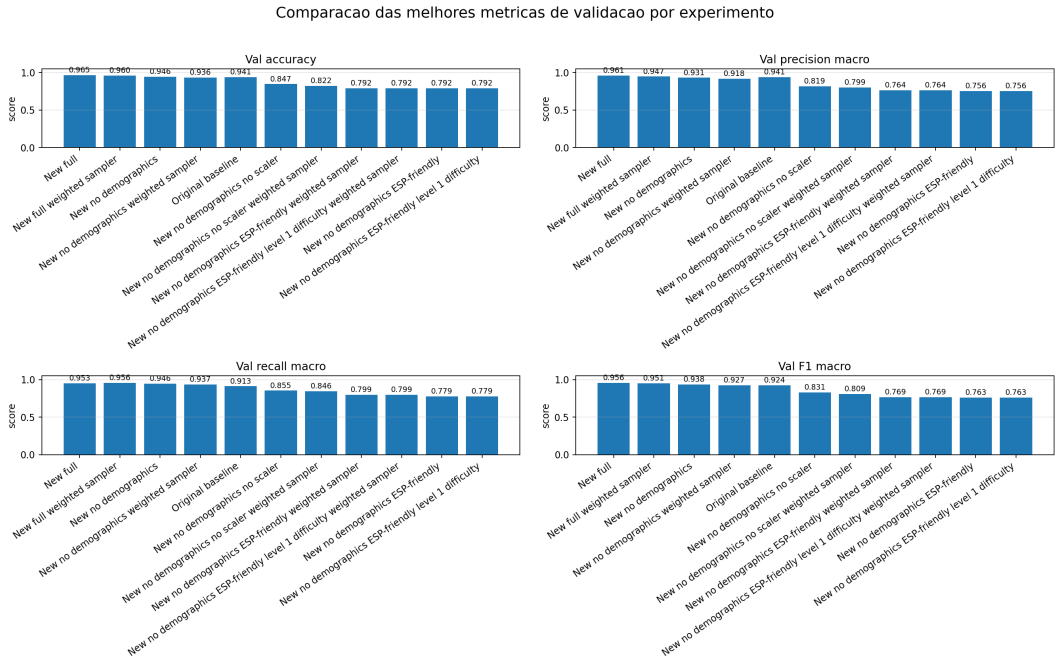


Figure 7. Comparison of core validation metrics at the best epoch selected by macro F1-score for each WESAD training configuration.

(0.1454), and its overfitting gap was small (0.0145) This indicates that the regularized TabularMLP improved local generalization compared with the simpler baseline, whose overfitting gap was 0.0728.

Removing demographic features produced a modest decrease when `StandardScaler` was still used: the macro F1-score moved from 0.9565 to 0.9378. This supports the privacy-oriented decision to exclude `age`, `height`, and `weight`; most of the discriminative signal remained in the physiological features.

The decisive engineering comparison, however, was not only the highest local F1-score. The model also needed a preprocessing contract that could be reproduced in the embedded and simulated data flow, local training clients, server-side validation, and mobile inference. Under this criterion, the selected integration model was *New no demo no scaler*. It reached 0.8307 macro F1 locally, below the scaled no-demographics model, but it avoids the requirement to broadcast and reproduce scaler statistics across all parts of the system. Weighted sampling did not improve this variant (0.8093), suggesting that class-weighted loss was sufficient for the current dataset.

The ESP-friendly models obtained lower macro F1-scores, between 0.7627 and 0.7693. This result suggests that the removed features, especially EDA decomposition and BVP peak frequency, carry useful classification signal. Nevertheless, these experiments remain important because they quantify the cost of simplifying the feature set for embedded computation.

Figure 9 reports the epoch at which each configuration achieved its highest validation macro F1-score. An epoch corresponds to one complete pass through the training set. The *New full* model reaches its optimum at epoch 77 and terminates training at epoch

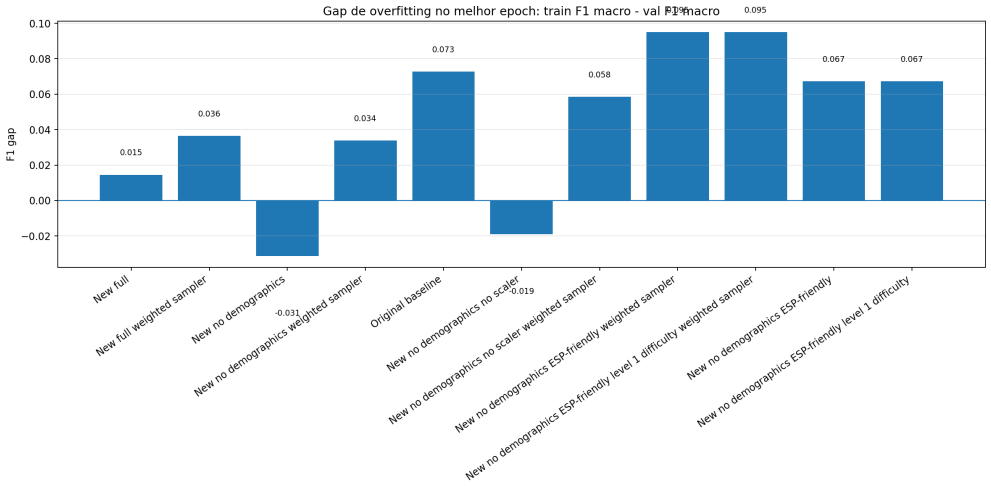


Figure 8. Overfitting gap, calculated as training F1 macro minus validation F1 macro, at the best epoch for each configuration. Lower values indicate better generalization; the comparison helps distinguish local validation performance from deployment-oriented model selection.

127 due to early stopping, demonstrating efficient convergence within roughly one quarter of the 300-epoch budget. The selected no-scaler model reaches its best F1 macro at epoch 117 and terminates at epoch 167, showing that removing standardization makes optimization harder but still viable. In contrast, the original baseline peaks as early as epoch 38 but at a substantially lower F1 value than the best scaled model, suggesting rapid memorization of the training set rather than robust learning. These timing differences are relevant for federated deployment: local training nodes at healthcare institutions benefit from models that converge reliably without excessive computational cost per federated round.

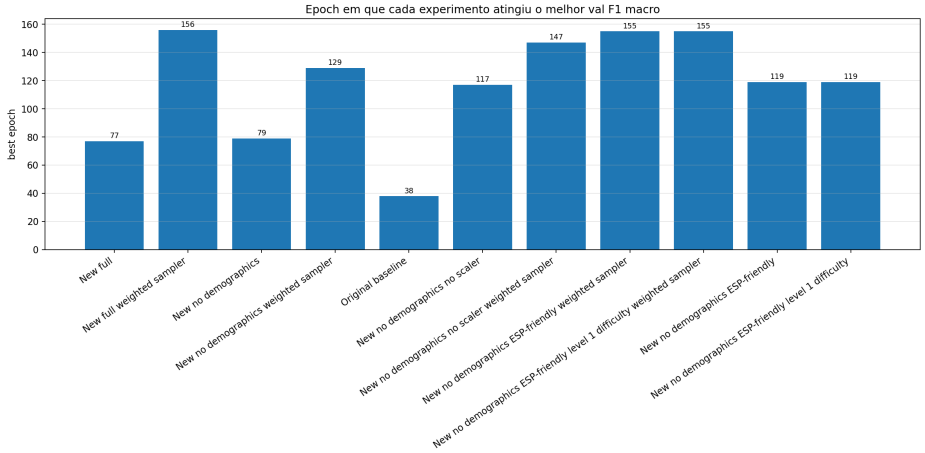


Figure 9. Best epoch selected by validation macro F1-score for each training configuration. Earlier peaks with higher F1, such as *New full* at epoch 77, indicate more efficient training; later peaks, such as weighted sampler variants, reflect slower convergence under aggressive class rebalancing.

Taken together, the experimental figures provide two complementary conclusions.

From a purely local validation perspective, the scaled *New full* TabularMLP achieves the best validation macro F1-score (0.9565), the lowest validation loss among the evaluated models (0.1454), and a small overfitting gap (0.0145), while converging in a moderate number of epochs. From a deployment perspective, the selected *New no demo no scaler* model is more appropriate for the federated prototype because it removes demographic attributes and avoids an external scaling dependency that was not consistently reproducible across embedded and simulated data paths, local training clients, server validation, and mobile inference. Thus, the project prioritizes a model with lower local F1 macro (0.8307) but a more reliable end-to-end preprocessing contract.

5.11. Integration with Federated Learning

The integration with federated learning was implemented by separating the local data ingestion service from the federated training process. This separation was an important architectural decision because the hospital-side HTTP API should not become responsible for coordinating training rounds, storing model weights, or uploading model files to a central endpoint. Instead, the local API only receives authenticated physiological windows and persists them in the local PostgreSQL database. The federated training responsibility is assigned to a Python service running in the same institutional environment, implemented as a Flower client. On the global side, a Flower server coordinates the rounds, aggregates the received model updates, and publishes the newest global model through a separate Global Model API. The overall organization of this implementation is illustrated in Figure 10, which provides a visual summary of how the mobile gateway, hospital-side ingestion services, local Flower clients, global Flower server, and model distribution API are connected.

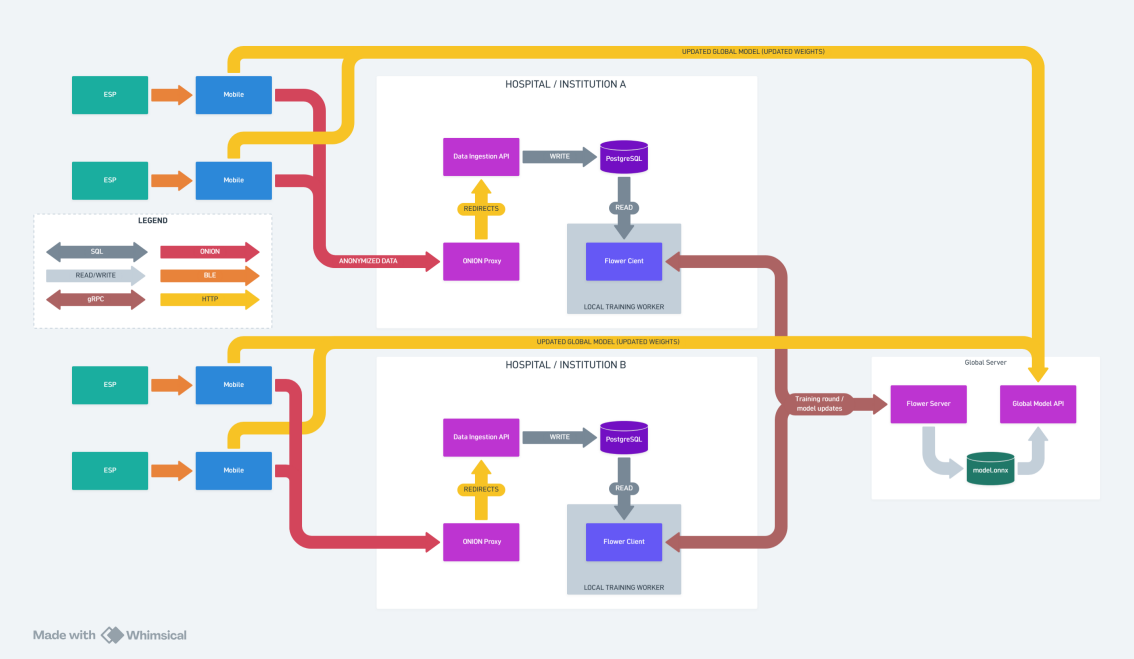


Figure 10. Implemented federated learning integration architecture, showing the separation between mobile data submission, hospital-side data ingestion, local Flower training, global aggregation, and model redistribution.

Table 13. Main software contracts in the federated learning integration.

Component	Input	Responsibility
Local API	HMAC-authenticated physiological windows.	Validates requests and persists accepted records in PostgreSQL.
PostgreSQL	Accepted sensor windows.	Stores institutional physiological records locally.
Local Flower client	Local database records and global parameters.	Trains the TabularMLP locally and returns updated model parameters and metrics.
Global Flower server	Client model updates.	Coordinates federated rounds and aggregates parameters using FedAvg.
Global Model API	Exported global model artifact and metadata.	Publishes the latest ONNX model and metadata for mobile retrieval.

5.11.1. Local Flower Client Integration at the Local Node

The local Flower client is deployed alongside the local API and the local PostgreSQL database. Its role is to transform the records accumulated by the API into a federated learning contribution. During each Flower round, the client receives the current global parameters from the server. These parameters are the numerical weights and biases of the neural network, not patient records. The client then reads the available rows from the local sensor table, maps textual labels such as `amusement`, `baseline`, and `stress` to numeric classes, and prepares the 26 physiological input features used by the privacy-oriented TabularMLP configuration. Demographic attributes are intentionally excluded from this federated contract, preserving the same privacy rationale evaluated in the WESAD experiments.

Before local optimization, the client performs preprocessing entirely inside the institutional boundary. Missing feature values are imputed using medians computed from the local training split, invalid labels are discarded, and the local dataset is divided into training and validation partitions when enough samples are available. For the integration path, this preprocessing deliberately avoids `StandardScaler`, so that the model receives features in the same numerical scale used by the embedded and simulated data paths and the server-side validation routine. The training routine follows the same model family validated experimentally: a multilayer perceptron with Batch Normalization, GELU activations, dropout, AdamW optimization, class-weighted cross-entropy, learning-rate scheduling, gradient clipping, and early stopping based on validation macro F1-score. The client returns only the updated model parameters and training metrics to Flower; the physiological records remain in the hospital database.

The implementation also handles operational edge cases required for a federated deployment. If the database contains no usable samples, or fewer samples than the minimum required for a meaningful local update, the client skips the training step and reports

a zero-example result with a controlled status. This behavior prevents an empty hospital node from corrupting the aggregation stage while still allowing the server to continue coordinating other participants. Therefore, this local training service acts as the bridge between the privacy-preserving ingestion pipeline and the federated coordination layer, without expanding the HTTP API beyond its data collection scope.

5.11.2. Global Flower Server Coordination and Aggregation

The global Flower server initializes the global TabularMLP model and listens for participating local clients. Communication between the server and clients uses Flower's federated training protocol over gRPC, a remote procedure call mechanism commonly used for communication between distributed services. Its coordination loop follows the standard federated learning pattern: the current global parameters are distributed to selected clients, each client trains locally using its private database, and the server receives the resulting parameter updates. The aggregation strategy is based on FedAvg, which computes a weighted average of client updates according to the number of local examples reported by each participant.

The global strategy was extended with two practical responsibilities. First, it filters ineffective rounds by skipping aggregation when all clients report zero examples, which can occur when no institution has accumulated enough local data. Second, after a successful aggregation, it materializes the global model lifecycle. The aggregated parameters are loaded back into the TabularMLP architecture, saved as a PyTorch checkpoint for traceability, evaluated when a validation dataset is available, exported to ONNX, and published as the latest model artifact. A checkpoint is a saved copy of the model state at a specific training round; it allows later inspection, rollback, or reexport. This lifecycle makes the federated server more than a transient coordinator: it becomes the source of versioned global models that can be audited, rolled back, or redistributed.

5.11.3. Global Model Distribution to the Mobile Application

The model redistribution path shown in Figure 10 is intentionally separated from the Flower training protocol. The mobile application does not communicate directly with the Flower server. This distinction is relevant because Flower uses a federated training protocol rather than a public model download API. For inference on the smartphone, the project introduces a Global Model API that exposes the latest exported model and its metadata through conventional HTTP endpoints. The endpoint `/model/latest` serves the current ONNX file, while `/model/latest/metadata` returns information such as the federated round, model identifier, file hash, file size, expected input dimension, feature names, output labels, participating clients, and aggregated metrics. The file hash allows the mobile application to verify whether the downloaded model matches the published metadata, and the feature list tells the application the exact order and meaning of the numerical inputs expected by the model.

Publishing the model in ONNX reduces coupling between the mobile application and the internal PyTorch representation used during training. The mobile side can download a stable inference artifact and validate its metadata before using it locally, while

the Flower server remains free to store internal checkpoints in PyTorch format. This design closes the federated loop proposed by the architecture: the mobile gateway contributes labeled physiological windows to the local institution, the local Flower client uses those records for local training, the global Flower server aggregates model updates across institutions, and the Global Model API returns the newest global model to the mobile application for local inference.

In the current prototype, the local WESAD experiments validate the quality and trade-offs of the model family that each local Flower client is expected to train, and the implemented Flower components define the operational path for federated rounds and global model publication. The selected integration model is not the highest-scoring local model; it is the no-demographics, no-scaler variant chosen because its preprocessing contract can be reproduced across the complete system. A full multi-institutional benchmark with independent hospital datasets remains a future evaluation step, but the integration already establishes the principal software contracts required for privacy-preserving federated learning: local-only data storage, parameter-based communication, global aggregation, versioned model export, and mobile-oriented model distribution.

5.12. Limitations of the Current Evaluation

Four limitations warrant explicit acknowledgment:

1. **Window-level split:** the WESAD train/validation split was performed at the window level rather than the subject level, meaning windows from the same participant may appear in both partitions. This may moderately inflate validation metrics relative to true out-of-subject generalization.
2. **No-scaler performance trade-off:** the selected no-scaler model achieved lower local validation performance than the best scaled model. This was an intentional engineering trade-off: removing `StandardScaler` reduced local macro F1-score but avoided a preprocessing dependency that was not reliably reproducible across embedded and simulated data paths, local training clients, server validation, and mobile inference.
3. **Controlled transmission tests:** the mobile-to-Tor transmission path was validated with controlled test payloads. Live integration with continuous ESP32 physiological streams remains an ongoing deployment step.
4. **No multi-institutional benchmark yet:** although the local Flower client and global Flower server components define the federated execution path, the present evaluation does not yet report a multi-institutional experiment with independent hospital nodes and heterogeneous local distributions.

Future work will address subject-level cross-validation, end-to-end latency characterization under Tor routing, and federated evaluation across multiple participating nodes.

6. Discussion of the Implications Discovered

The practical results reinforce the central thesis of this research: occupational stress monitoring can be pursued through a layered privacy architecture that combines federated learning with complementary anonymization and authentication mechanisms. Four implications emerge from the implementation.

First, the prototype shows that privacy preservation must begin before data reach the smartphone. The Bluetooth sensing channel was not treated as a neutral transport link; instead, it was constrained by authenticated bonding, persistent ownership, and authorization checks before physiological records or acknowledgement commands could be exchanged.

Second, delegating Tor connectivity to the mobile gateway resolves a fundamental tension between privacy and hardware constraints. Embedded wearables remain responsible for data acquisition, while the smartphone, already present in occupational settings, assumes the role of anonymized communication endpoint. This division of labor is scalable and avoids overburdening resource-limited microcontrollers.

Third, the combination of Tor and HMAC demonstrates that network anonymity and application-layer authentication address distinct threat models and must coexist. Tor alone would allow any anonymous actor to submit data to the hidden service; HMAC alone would authenticate requests but expose the client's network identity. Their joint deployment aligns with the defense-in-depth principle advocated in the federated learning literature reviewed in Section 3.

Fourth, the classification experiments indicate that a regularized tabular MLP achieves strong stress detection performance on window-aggregated physiological features. The scaled privacy-oriented variant, excluding demographic attributes, retained over 98% of the full model's macro F1-score, while the selected no-scaler variant traded part of this local performance for a preprocessing contract that can be reproduced across embedded and simulated data paths, local training clients, server validation, and mobile inference. This finding supports the feasibility of deploying local models at healthcare institutions without relying on quasi-identifying attributes, while also showing that deployment consistency must be considered alongside validation metrics.

7. Conclusion

This study investigated federated learning as a privacy-preserving approach to occupational stress analysis, combining a systematic literature review with a practical prototype spanning wearable sensing, mobile anonymized communication, local model training, and federated coordination The literature review confirmed that federated learning, differential privacy, and secure aggregation techniques are central to protecting sensitive physiological data in distributed health applications The practical implementation evolved from an ESP32-centric gateway model to a mobile-mediated architecture, in which authenticated Bluetooth bonding restricts local sensor access to the enrolled smartphone, Tor provides topological anonymity, and HMAC-SHA256 ensures authenticated, tamper-evident data submission to local training nodes hosted at healthcare institutions.

Experimental validation on the WESAD dataset demonstrated that the scaled TabularMLP classifier achieves a validation macro F1-score of 0.9565 for three-class affective state classification, outperforming a simpler baseline while maintaining low overfitting. For the federated prototype, the selected no-demographics, no-scaler variant prioritizes reproducible preprocessing across the embedded sensing, simulated data, local training, server validation, and mobile inference paths. Future work will complete live ESP32 stream integration, subject-level cross-validation, Tor latency evaluation, and multi-institutional federated training rounds.

References

[Alahmadi et al. 2023] Alahmadi, A., Khan, H. A., Shafiq, G., Ahmed, J., Ali, B., Javed, M. A., Khan, M. Z., Alsisi, R. H., and Alahmadi, A. (2023). A privacy-preserved iomt-based mental stress detection framework with federated learning. *The Journal of Supercomputing*, 80:10255–10274.

[Alhejaili and Alomainy 2023] Alhejaili, R. and Alomainy, A. (2023). The use of wearable technology in providing assistive solutions for mental well-being.

[Almadhor et al. 2023] Almadhor, A., Sampedro, G. A., Abisado, M., Abbas, S., Kim, Y., Khan, M. A., Baili, J., and Cha, J. (2023). Wrist-based electrodermal activity monitoring for stress detection using federated learning. *Sensors*, 23:3984–3984.

[Alshamrani 2021] Alshamrani, M. (2021). An advanced stress detection approach based on processing data from wearable wrist devices. *International Journal of Advanced Computer Science and Applications*, 12.

[Benouis et al. 2023] Benouis, M., Can, Y. S., and André, E. (2023). A privacy-preserving multi-task learning framework for emotion and identity recognition from multimodal physiological signals.

[Bharathi et al. 2024] Bharathi, M., Srinivas, T. A. S., and Bhuvaneswari, M. (2024). Federated learning: From origins to modern applications and challenges. *Journal of Information Technology and Cryptography.*, 1:29–38.

[Can and Ersoy 2021] Can, Y. S. and Ersoy, C. (2021). Privacy-preserving federated deep learning for wearable iot-based biomedical monitoring. *ACM Transactions on Internet Technology*, 21:1–17.

[Chen et al. 2021] Chen, J., Abbod, M., and Shieh, J.-S. (2021). Pain and stress detection using wearable sensors and devices—a review.

[Dai et al. 2021] Dai, S., Meng, F., Wang, Q., and Chen, X. (2021). Federatednilm: A distributed and privacy-preserving framework for non-intrusive load monitoring based on federated deep learning. *arXiv (Cornell University)*.

[Fauzi et al. 2022] Fauzi, M. A., Yang, B., and Blobel, B. (2022). Comparative analysis between individual, centralized, and federated learning for smartwatch based stress detection. *Journal of Personalized Medicine*, 12:1584–1584.

[Gafni et al. 2022] Gafni, T., Shlezinger, N., Cohen, K., Eldar, Y. C., and Poor, H. V. (2022). Federated learning: A signal processing perspective. *IEEE Signal Processing Magazine*, 39:14–41.

[Gahlan and Sethia 2023] Gahlan, N. and Sethia, D. (2023). Federated learning inspired privacy sensitive emotion recognition based on multi-modal physiological sensors. *Cluster Computing*, 27:3179–3201.

[Huang et al. 2024] Huang, H., Iskandarov, B., Rahman, M., Otal, H. T., and Canbaz, M. A. (2024). Federated learning in adversarial environments: Testbed design and poisoning resilience in cybersecurity. *arXiv (Cornell University)*.

[Jiang et al. 2023] Jiang, S., Firouzi, F., and Chakrabarty, K. (2023). Low-overhead clustered federated learning for personalized stress monitoring. *IEEE Internet of Things Journal*, 11:4335–4347.

[Jiang et al. 2024] Jiang, S., Li, Y., Firouzi, F., and Chakrabarty, K. (2024). Federated clustered multi-domain learning for health monitoring. *Scientific Reports*, 14.

[Johnson and Phillips 2018] Johnson, N. and Phillips, M. (2018). Rayyan for systematic reviews. *Journal of Electronic Resources Librarianship*, 30.

[Konečný et al. 2016] Konečný, J., McMahan, H. B., Ramage, D., and Richtárik, P. (2016). Federated optimization: Distributed machine learning for on-device intelligence. *arXiv (Cornell University)*.

[Li et al. 2020] Li, T., Sahu, A. K., Talwalkar, A., and Smith, V. (2020). Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine*, 37:50–60.

[Lin et al. 2024] Lin, P.-C., Li, J.-L., Chien, W.-S., and Lee, C.-C. (2024). In-the-wild physiological-based stress detection using federated strategy. pages 1681–1685.

[Liu et al. 2021] Liu, J. C., Goetz, J., Sen, S., and Tewari, A. (2021). Learning from others without sacrificing privacy: Simulation comparing centralized and federated machine learning on mobile health data.

[Luong et al. 2023] Luong, T. D., Tien, V. M., Quyen, N. H., Hien, D. T. T., Duy, P. T., and Pham, V.-H. (2023). Fed-lsae: Thwarting poisoning attacks against federated cyber threat detection system via autoencoder-based latent space inspection. *arXiv (Cornell University)*.

[Ma et al. 2020] Ma, C., Li, J., Ding, M., Yang, H. H., Shu, F., Quek, T. Q. S., and Poor, H. V. (2020). On safeguarding privacy and security in the framework of federated learning. *IEEE Network*, 34:242–248.

[Mariotti 2015] Mariotti, A. (2015). The effects of chronic stress on health: New insights into the molecular mechanisms of brain–body communication.

[Quick and Henderson 2016] Quick, J. C. and Henderson, D. (2016). Occupational stress: Preventing suffering, enhancing wellbeing. *International Journal of Environmental Research and Public Health*, 13:459–459.

[Sadilek et al. 2021] Sadilek, A., Liu, L., Nguyen, D. T., Kamruzzaman, M., Serghiou, S., Rader, B., Ingerman, A., Mellem, S., Kairouz, P., Nsoesie, E. O., MacFarlane, J., Vullikanti, A., Marathe, M., Eastham, P. R., Brownstein, J. S., y Arcas, B. A., Howell, M., and Hernandez, J. (2021). Privacy-first health research with federated learning. *npj Digital Medicine*, 4.

[Spector 2002] Spector, P. E. (2002). Employee control and occupational stress. *Current Directions in Psychological Science*, 11:133–136.

[Vyas et al. 2023] Vyas, J., Bhumika, B., Das, D., and Chaudhury, S. (2023). Dr. mtl: Driver recommendation using federated multi-task learning. *2021 IEEE 94th Vehicular Technology Conference (VTC2021-Fall)*, pages 1–5.

[Wang et al. 2024] Wang, Z., Yang, Z., Azimi, I., and Rahmani, A. M. (2024). Differential private federated transfer learning for mental health monitoring in everyday settings: A case study on stress detection. *arXiv (Cornell University)*.

[Xu et al. 2020] Xu, J., Glicksberg, B. S., Su, C., Walker, P., Bian, J., and Wang, F. (2020). Federated learning for healthcare informatics. *Journal of Healthcare Informatics Research*, 5:1–19.